Abstraction:

```java
package Oops;

// Abstract Parent Class
abstract class PaymentGateway {
    // Concrete method (shared behavior)
    void printReceipt() {
        System.out.println("Receipt generated.");
    }

    // Abstract method: NO body, child MUST implement this
    abstract void processPayment(double amount);
}

// Concrete Child Class 1
class UPIPayment extends PaymentGateway {
    @Override
    void processPayment(double amount) {
        System.out.println("Processing ₹" + amount + " via UPI QR code.");
    }
}

// Concrete Child Class 2
class CreditCardPayment extends PaymentGateway {
    @Override
    void processPayment(double amount) {
        System.out.println("Processing ₹" + amount + " via Card Swipe and
OTP.");
    }
}

public class abstraction {
    public static void main(String[] args) {
        // PaymentGateway p = new PaymentGateway(); // ERROR: Cannot
instantiate abstract class!

        PaymentGateway payment = new UPIPayment();
        payment.processPayment(250.0); // Child decides the "how"
        payment.printReceipt();
    }
}
```

Encapsulation

```java
package Oops;

class BankAccount1 {
    // 1. Private fields (hidden from outside)
    private String holderName;
    private double balance;

    public BankAccount1(String holderName, double balance) {
        this.holderName = holderName;
        setBalance(balance); // Use setter for validation
    }

    // Getter (read-only access)
    public double getBalance() {
        return this.balance;
    }

    // Setter (controlled write access with validation)
    public void setBalance(double amount) {
        if (amount >= 0) {
            this.balance = amount;
        } else {
            System.out.println("Invalid balance: cannot be negative!");
        }
    }
}

public class encapsulation {
    public static void main(String[] args) {
        BankAccount1 acc = new BankAccount1("Alex", 500);

        // acc.balance = -100; // COMPILER ERROR: balance has private access!
        acc.setBalance(-100);  // Prints: Invalid balance: cannot be negative!
        System.out.println(acc.getBalance()); // 500.0
    }
}
```

Polymorphism

```java
package Oops;

//compiletime

class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    // Same name, three parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Same name, different types
    double add(double a, double b) {
        return a + b;
    }
}
// RunTime
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks: Woof woof!");
    }
}



class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Cat meows: Meow meow!");
    }
}

public class polymorphism {
    public static void main(String[] args) {
        // Parent reference pointing to child objects
        Animal pet1 = new Dog();
        Animal pet2 = new Cat();
        Animal pet3 = new Animal();
```

```
        pet1.makeSound(); // Prints: Dog barks: Woof woof!
        pet2.makeSound(); // Prints: Cat meows: Meow meow!
        pet3.makeSound();
    }
}
```

Inheritance

```
package Oops;

// Parent Class (Superclass)
class Vehicle {
    // ttribute
    String brand;

    // method
    void startEngine() {
        System.out.println(brand + " engine started.");
    }
}

// Child Class (Subclass) inherits from Vehicle
class Bike extends Vehicle {
    boolean hasCarrier;

    void kickStand() {
        System.out.println("Kickstand put down.");
    }
}

public class inheritance {
    public static void main(String[] args) {
        Bike myBike = new Bike();
        myBike.brand = "shine";     // Inherited from Vehicle
        myBike.startEngine();        // Inherited from Vehicle
        myBike.kickStand();          // Bike's own method
    }
}
```